



Høgskulen
på Vestlandet



DAT108

Programmering og webapplikasjoner

F03 – Mer om lambda-uttrykk

Lars-Petter Helland
Bergen
24. august 2026

Ressurser til i dag

Flere videoer, ca. 45 minutter:

Navn	Lengde
 Java 8 Lambda Basics 13 - Lambda Exercise.mp4	00:04:04
 Java 8 Lambda Basics 14 - Lambda Exercise Solution.mp4	00:17:31
 Java 8 Lambda Basics 15 - Using Function Interfaces.mp4	00:07:05
 Java 8 Lambda Basics 16 - More Functional Interfaces.mp4	00:06:22
 Java 8 Lambda Basics 19 - Closures in Lambda Expressions.mp4	00:08:29
 Java 8 Lambda Basics 21 - Method References.mp4	00:10:26

>>> [Lenke til video nr 13](#)



Ressurser til i dag, forts ..

Lesestoff / pensum:

Fra <https://dev.java/learn/lambda/>

- › <https://dev.java/learn/lambda/first-lambda/>
- › <https://dev.java/learn/lambda/functional-interfaces/>
- › <https://dev.java/learn/lambda/method-references/>

(alternativt) fra <https://www.baeldung.com/>

- › <https://www.baeldung.com/java-8-functional-interfaces>
- › <https://www.baeldung.com/java-method-references>



Plan for dagen

Jeg har lagt opp til at vi kan gjøre eksemplene sammen i dag!

Utgangspunkt for alle eksemplene er i prosjektet **f03-recapf02-people-fyllinn**.

Recap/ fikse / gjøre ferdig eksempler fra forrige gang f02:

- › Heltallsfunksjon (`no.hvl.dat108.f02.funksjoner`) recap + **bruke lambda-uttrykk + i variabel**
- › Utvalgssortering (`no.hvl.dat108.f02.sortering1`) recap + **default sammenligning**
- › `Collections.sort` (`no.hvl.dat108.f02.sortering2`) recap
- › Tallplukker (`no.hvl.dat108.f02.utplukk`) recap + **metode som returnerer en funksjon**

Nytt lite eksempel (gjøre sammen) Betinget utskrift (`no.hvl.dat108.f03.betingetutskrift`)

Litt mer “teori”: syntaks for lambda-uttrykk, metode som returnerer en funksjon, `@FunctionalInterface`, metodereferanser

Nytt eksempel med en liste av personer. Jobbe med dette i 2. time.



Recap/ fikse / gjøre ferdig eksempler fra forrige gang f02

- › GitHub <https://github.com/lars-petter-helland/dat108-h2025>
- › Prosjekt **f03-recapf02-people-fyllinn**
 - › Heltallsfunksjon (`no.hv1.dat108.f02.funksjoner`)
 - › Utvalgssortering (`no.hv1.dat108.f02.sortering1`)
 - › Collections.sort (`no.hv1.dat108.f02.sortering2`)
 - › Tallplukker (`no.hv1.dat108.f02.utplukk`)



- › Etter timen legges det ut et løsningsprosjekt i GitHub med navnet **f03-recapf02-people-losning**.

Main takeaways

fra Heltallsfunksjon (no.hv1.dat108.f02.funksjoner)

- › Det kan være nyttig å bruke en “hjelpfunksjon” til å gjøre en liten del av det en metode skal gjøre. Dette gjør metoden mer generell / fleksibel.
- › En slik hjelpfunksjon kan da gis inn som en parameter til metoden, og brukes på ønsket sted.
- › I Java har vi ikke funksjoner direkte, men bruker objekter (av et interface) med én metode i stedet.
- › Hvis et interface kun har én metode (kalles da et **funksjonelt interface**), kan et objekt av denne interface-typen opprettes med et **lambda-uttrykk**, og evt. refereres av en variabel. F.eks:

```
Heltallsfunksjon kvadrat = x -> x * x;  
omform(listen, kvadrat);
```



Main takeaways

fra Utvalgssortering (no.hvl.dat108.f02.sortering1)

- › Vår Utvalgssortering ble gjort mer fleksibel ved å legge til en ekstra parameter som sier hvordan objektene skal sammenlignes / sorteres. Ingredienser:

```
interface Sammenligner<T> {  
    int sammenlign(T a, T b);  
}  
  
void sorter(List<T> liste, Sammenligner<T> sammenligner) {  
    if (sammenligner.sammenlign(a, b) < 0) ...
```

- › Vi så også at vi kunne beholde “standard” sortering ved å bruke T sin `compareTo()` som sammenligner, slik at selve sorteringen kan gjøres i den generelle metoden. (slipper duplikatkode)



Main takeaways

fra Collections.sort (no.hvl.dat108.f02.sortering2)

- › Vi så at tilsvarende gjøres i Java sin Collection.sort(), men da med:

```
interface Comparator<T> {  
    int compare(T a, T b);  
}  
  
void sort(List<T> liste, Comparator <T> comparator) {  
    if (comparator.compare(a, b) < 0) ...
```

- › Vi kan da med fordel bruke Comparator i stedet for vår hjemmelagde Sammenligner.



Main takeaways

fra Tallplukker (no.hvl.dat108.f02.utplukk)

- › Vi så tidligere hvordan funksjonsuttrykk (lambdauttrykk) kunne brukes som parameter til en metode.
- › Nå så vi også hvordan en metode kan returnere et funksjonsuttrykk (lambdauttrykk), og vi kan på den måten få "dynamiske" lambdauttrykk.
- › Eksempel

```
public static Betingelse størreEnn(int tall) {  
    return x -> x > tall;  
}
```
- › Funksjoner som tar funksjoner som parametre, eller som returnerer funksjoner, kalles høyere ordens funksjoner.



Gjøre Oppgave 4 (BetingetUtskrift) fra Oppgaver_uke34

- › GitHub <https://github.com/lars-petter-helland/dat108-h2025>
- › Prosjekt **f03-recapf02-people-fyllinn**
- › Pakke **no.hvl.dat108.f03.betingetutskrift**



DAT108 Funksjoner og lambda-uttrykk

Lars-Petter Helland, 17.08.2026

Uke34 - Oppgave 4

```
/*
 * Her er en liten oppgave dere kan prøve på labben og frem til
 * mandag for å se om dere har fått med dere hovedideen med
 * lambda-uttrykk.
 *
 * Oppgavetekst:
 *
 * Lag en generell metode betingetUtskrift(...) som tar inn en streng
 * og et (false | true) lambda-uttrykk og som skriver ut strengen kun
 * hvis uttrykket er sant.
 *
 * Lag en main(...) -metode som tester dette ut med et par ulike input,
 * f.eks. krav om at stringen begynner på "X" eller at den er lengre
 * enn 3 tegn.
 */
public class OppgaveBetingetUtskrift {
    private static void betingetUtskrift(String s, ???) {
        ???
    }
    public static void main(String[] args) {
        ???
    }
}
```



Nå litt generell «teori» før vi tar flere eksempler

- › Hva er en funksjon i Java?
- › Hvordan ser lambda-uttrykk ut?
- › Kan vi bruke eksisterende metoder som funksjoner?



Hva er en funksjon i Java

Hvordan er det implementert?

I Java er funksjoner implementert som objekter. En funksjon er et **objekt** (av en **klasse**) som implementerer et **interface** (en kontrakt) som har én metode.

Å utføre funksjonen blir da det samme som å **utføre denne ene metoden** i objektet.

```
List<Integer> kvadratListe = omform(listen, new Kvadrat());
```

```
class Kvadrat implements HeltallsFunksjon {  
    @Override  
    public int av(int x) { return x*x; }  
}
```

```
interface HeltallsFunksjon {  
    int av(int x);  
}
```

```
static List<Integer> omform(List<Integer> liste, HeltallsFunksjon fu) {  
    List<Integer> nyListe = new ArrayList<>();  
    for (int tall : liste) {  
        nyListe.add(fu.av(tall));  
    }  
}
```

Funksjonell kontrakt og @FunctionalInterface

Vi har nå i introduksjonen sett at vi kan definere våre egne funksjonstyper (interfaces) som Heltallsfunksjon, Sammenligning og Betingelse.

En funksjonell kontrakt har kun én metode!

Det er vanlig at interfaces som er tenkt brukt som funksjoner annoteres med @FunctionalInterface. Dette sikrer at vi ikke ved et uhell senere legger til flere metoder, og på den måten lager kluss i koden vår.

```
@FunctionalInterface
interface HeltallsFunksjon {
    int av(int x);
}
```

Innebyggede funksjonstyper i Java

Hvis vi bruker generics, kan vi dekke de fleste behov for funksjonstyper med noen få interfaces. Java kommer med de interfacene vi trenger, og vi trenger derfor ikke å definere våre egne. F.eks. kan vi bruke `Function<T,U>`

```
Function<Integer, Integer> kvadrat = x -> x * x;
```

i stedet for Heltallsfunksjon

```
@FunctionalInterface
interface HeltallsFunksjon {
    int av(int x);
}
```

Innebyggede funksjonelle kontrakter i Java

Vi finner en rekke funksjonelle kontrakter i pakken `java.util.function`

Disse må dere kunne!

- › `Function<T,R>` `Function<String,Integer> stringlengde = s -> s.length();`
- › `Consumer<T>` `Consumer<String> skrivUt = s -> System.out.println(s);`
- › `BiFunction<T,U,R>` `BiFunction<Integer, Integer, Double> snitt = (a,b) -> (a+b)/2.0;`
- › `Predicate<T>` `Predicate<Person> erMyndig = p -> p.getAge() >= 18;`
- › `UnaryOperator<T>` `UnaryOperator<String> tilSmaa = s -> s.toLowerCase();`
- › `BinaryOperator<T>` `BinaryOperator<Integer> denStorste = (a,b) -> Math.max(a,b);`
- › `Supplier<T>` `Supplier<Double> tilfeldig0til10 = () -> Math.random() * 10;`

Det finnes noen flere. Vi bruker stort sett alltid en av de innebyggede kontraktene.



Hva representerer et lambda-uttrykk?

Et lambda-uttrykk er en syntaks for å representere en **anonym funksjon**.

I Java (siden funksjoner i Java er objekter) vil et lambda-uttrykk da være et uttrykk for et objekt (av en klasse) som implementerer et `@FunctionalInterface`.

Eksempel:

```
(int x) -> x * x
```

er et objekt som implementerer `Function<Integer, Integer>`-interfacet.

Dette lambda-uttrykket kan da f.eks. brukes som parameter der det forventes en `Function<Integer, Integer>`, og det kan lagres i en variabel av typen `Function<Integer, Integer>`.



Lambda-uttrykk, noen eksempler

```
(int x) -> 2 * x           // Parameter av type int. Returnerer en int.  
(a, b) -> (a + b)/2.0     // Parameter av type int/double/?. Returnerer en double.  
s -> System.out.println(s) // Parameter av type ?. Skriver ut s. Returnerer ingenting (void).  
() -> System.out.println("Hei") // Ingen parameter. Skriver ut noe. Returnerer ingenting (void).  
  
(String s) -> {           // Hvis funksjonen vår er mer enn en setning, må  
    if (s != null && s.equals("Bingo")) { // den puttes i en blokk {}, og siste setning må  
        System.out.println(s);           // være en return (hvis den skal returnere noe).  
    }  
    return s.toUpperCase();  
}
```

18



Lambda-uttrykk og funksjonstyper

Øvelse

Hvilken funksjonstype(r) passer til de ulike uttrykkene?

- › `(int x) -> 2 * x`
- › `(a, b) -> (a + b)/2.0`
- › `x -> x > 5`
- › `s -> System.out.println(s)`
- › `() -> System.out.println("Hei")`
- › `(String s) -> {
 if (s != null && s.equals("Bingo")) {
 System.out.println(s);
 }
 return s.toUpperCase();
}`

Kan vi bruke eksisterende metoder som funksjoner?

Hva hvis vi har funksjoner (lambda-uttrykk) som rett og slett utfører metodekall på parametrene, kan vi forenkle disse ved kun å oppgi metodenavnet !

Dette kalles en **metodereferanse**, og foretrekkes ofte fremfor lambda-uttrykk

Eksempel: Lambda-uttrykket

```
s -> System.out.println(s)
```

kan skrives som

```
System.out::println
```

Vanlige eksempler på metodereferanser

Static metode:

`(a,b) -> Math.max(a,b)` kan skrives som `Math::max`

`(x) -> Math.abs(x)` kan skrives som `Math::abs`

Instansmetode generelt:

`s -> s.toUpperCase()` kan skrives som `String::toUpperCase`

`(s1,s2) -> s.compareToIgnoreCase()` kan skrives som `String::compareToIgnoreCase`

Instansmetode spesifikt objekt:

`x -> myObject.doSomething(x)` kan skrives som `myObject::doSomething`

Konstruktør:

`() -> new ArrayList()` kan skrives som `ArrayList::new`



Nytt eksempel med en liste av personer

Dette skal vi bruke en del tid på sammen ...

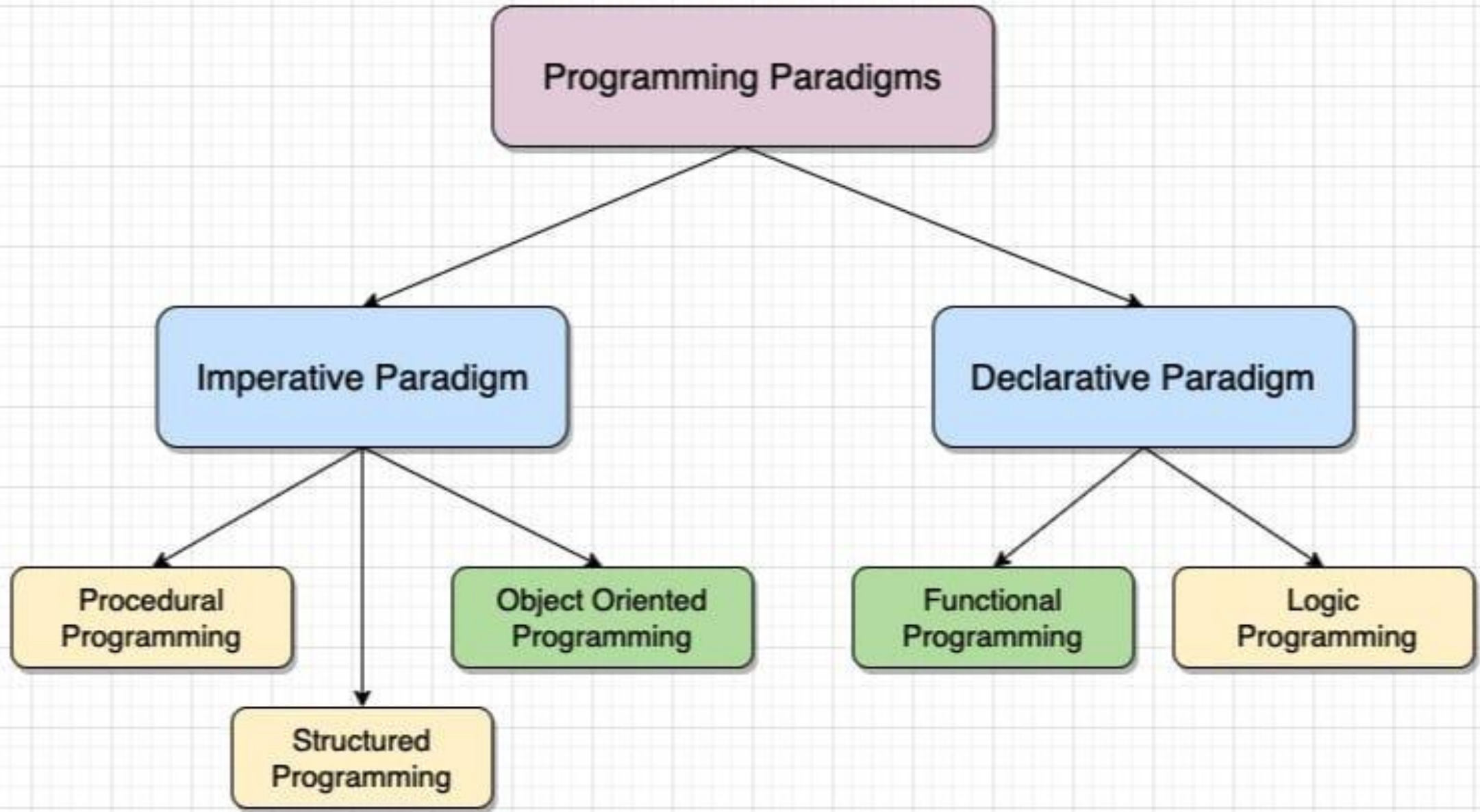
- › GitHub <https://github.com/lars-petter-helland/dat108-h2025>
- › Prosjekt **f03-recapf02-people-fyllinn**
- › Pakke **no.hvl.dat108.f03.people**



- › Etter timen legges det ut et løsningsprosjekt i GitHub med navnet **f03-recapf02-people-losning**.

Til avrunding ...





Imperativ vs. funksjonell programmering

- › **Imperativ stil:** En sekvens av setninger som via endringer i programmets tilstand (variabler) arbeider seg frem til et svar.

```
List<Integer> positiveTall = new ArrayList<>();  
for (int x : enListeMedTall) {  
    if (x > 0) {  
        positiveTall.add(x);  
    }  
}
```

- › **Funksjonell stil:** Nøstede funksjonskall som gir et svar direkte.

```
List<Integer> positiveTall = enListeMedTall .. filter(x -> x > 0) ..;
```

Trenger vi egentlig funksjonell programmering?

Noen fordeler med funksjonell vs. imperativ programmering er at vi får:

- › Kode som er mer **fleksibel** (vi kan parametrisere oppførsel)
- › Kode som er mer **lesbar** (høyere abstraksjonsnivå, færre kodelinjer)
- › Kode som er lettere å **vedlikeholde** (færre kodelinjer, bedre modularitet)
- › Kode som er lettere å **teste** (færre variabler, ingen sideeffekter)
- › Kode som er lettere å **parallellisere** (kommer tilbake til det)



Neste gang: streams

- › Neste gang skal vi se på det som kalles Java sitt Stream-API.
- › Dette gir oss kraftige verktøy for å jobbe med lister eller sekvenser av data
- › Vi skal se at programmer der dere tidligere har brukt 10 kodelinjer nå kan skrives med én enkel kodelinje.
- › Ved å kombinere noen standard metodekall og bruke lambda-uttrykk kan vi skrive programmer på et høyere abstraksjonsnivå, nesten som i SQL. F.eks.:

```
List<String> etternavnene = people.stream()           //FROM people
    .filter(p -> p.getAge() > 50)                   //WHERE age > 50
    .map(Person::getLastName)                        //SELECT lastName
    .toList();
```



Videre

- Ukeoppgaver uke 34 (hvis dere ikke er ferdig med disse) og uke 35
- Torsdag: Javahjelp
- Fredag. Forelesning om:
 - › Jobbe litt videre med people-eksempelet
 - › `Comparator.comparing()`
 - › Streams API-et. Pensumstoff:
 - › Video 22-24
 - › <https://www.youtube.com/watch?v=t1-YZ6bF-g0>
 - › <https://stackify.com/streams-guide-java-8/>
- Oblig0. Frist fredag 28. august. Grupper på 2-4.
- Oblig1. Frist søndag 6. september. Grupper på 2-4.

